LIST OF EXPERIMENTS

| | |
|---|---|
| 1. | PROGRAMS USING DATA FRAMES |
| 2. | PROGRAMS USING FUNCTIONS AND FILES |
| 3. | PROGRAMS USING NUMPY |
| 4. | PROGRAMS TO SOLVE LINEAR ALGEBRA PROBLEMS WITH NUMPY FUNCTIONS |
| 5. | PROGRAMS USING PANDAS |
| 6. | SOLVING LINEAR ALGEBRA PROBLEMS WITH PANDAS FUNCTIONS |
| 7. | PROGRAMS FOR READING AND WRITING DATA IN TEXT AND BINARY FORMATS |
| 8. | PROGRAMS FOR INTERACTING WITH DATABASES |
| 9. | PROGRAMS FOR INTERACTING WITH WEB APIS |
| 10. | PROGRAMS FOR STRING MANIPULATIONS |
| 11. | DATA WRANGLING |
| 12. | DATA AGGREGATION OPERATIONS |
| 13. | HANDLE TIME SERIES DATA |
| 14. | VISUALIZATION OF DIFFERENT KINDS OF DATA |
| 15. | PROGRAMS FOR DISTRIBUTION ANALYSIS |

| EX NO:1 | PROGRAMS USING DATA FRAMES |
|---|---|
| DATE: | |

**AIM:**

To understand and implement basic operations using DataFrames in Python with the help of the pandas library.

**ALGORITHM:**

STEP 1: Import the pandas library.
STEP 2: Create a DataFrame using a dictionary.
STEP 3: Display the DataFrame.
STEP 4: Perform basic operations:

- Access specific columns
- Add a new column
- Delete a column
- Filter data based on conditions
- Perform statistical operations (mean, maximum, minimum)

STEP 5: Print the results of each operation.

**PROGRAM:**

```
# Import pandas

import pandas as pd

# Create a DataFrame

data = {

    'Name': ['Alice', 'Bob', 'Charlie', 'David'],

    'Age': [24, 27, 22, 32],

    'Score': [85, 90, 78, 88]

}

df = pd.DataFrame(data)

# Display the DataFrame

print("Original DataFrame:")

print(df)

# Access specific column

print("\nNames of all students:")

print(df['Name'])
```

```
# Add a new column
df['Grade'] = ['B', 'A', 'C', 'B']
print("\nDataFrame after adding 'Grade' column:")
print(df)
# Delete a column
df.drop('Score', axis=1, inplace=True)
print("\nDataFrame after deleting 'Score' column:")
print(df)
# Filter data
print("\nStudents aged above 25:")
print(df[df['Age'] > 25])
# Statistical operations
print("\nAverage Age of Students:", df['Age'].mean())
print("Maximum Age:", df['Age'].max())
print("Minimum Age:", df['Age'].min())
```

**OUTPUT:**

```
Original DataFrame:
      Name  Age  Score
0    Alice   24     85
1      Bob   27     90
2  Charlie   22     78
3    David   32     88

Names of all students:
0      Alice
1        Bob
2    Charlie
3      David
Name: Name, dtype: object

DataFrame after adding 'Grade' column:
      Name  Age  Score Grade
0    Alice   24     85     B
1      Bob   27     90     A
2  Charlie   22     78     C
3    David   32     88     B

DataFrame after deleting 'Score' column:
      Name  Age Grade
0    Alice   24     B
1      Bob   27     A
2  Charlie   22     C
3    David   32     B

Students aged above 25:
    Name  Age Grade
1    Bob   27     A
3  David   32     B

Average Age of Students: 26.25
Maximum Age: 32
```

**RESULT:**

Thus, the program successfully demonstrates the creation and manipulation of DataFrames using pandas. Operations such as column access, addition, deletion, filtering, and statistical analysis were performed and verified.

| EX NO:2 | PROGRAMS USING FUNCTIONS AND FILES |
|---|---|
| DATE: | |

**AIM:**

To write a Python program that demonstrates the use of user-defined functions and file handling operations such as writing to and reading from a file.

**ALGORITHM:**

STEP 1: Define a function to accept user input and return a formatted string.
STEP 2: Define another function to write the string to a file.
STEP 3: Define a third function to read the contents of the file and display it.
STEP 4: In the main block, call the functions in sequence:
- Get input from the user.
- Write the data to a file.

STEP 5: Read and display the file contents.

**PROGRAM:**

Python
```
# Function to get user input
def get_user_data():
    name = input("Enter your name: ")
    age = input("Enter your age: ")
    return f"Name: {name}, Age: {age}\n"

# Function to write data to a file
def write_to_file(data, filename):
    with open(filename, 'w') as file:
        file.write(data)

# Function to read data from a file
def read_from_file(filename):
    with open(filename, 'r') as file:
        content = file.read()
        print("\nContents of the file:")
        print(content)

# Main block
def main():
    filename = "user_data.txt"
    user_data = get_user_data()
    write_to_file(user_data, filename)
    read_from_file(filename)

# Call main
main()
```

**OUTPUT:**

```
Enter your name: ARUN
Enter your age: 30

Contents of the file:
Name: ARUN, Age: 30
```

**RESULT:**

The program successfully demonstrates the use of functions for modular programming and file handling operations in Python. It collects user input, writes the data to a file, and reads the file content back for display

| EX NO:3 | PROGRAMS USING NUMPY |
|---|---|
| DATE: | |

**AIM:**

To implement basic operations using the NumPy library in Python, including array creation, reshaping, slicing, and mathematical computations.

**ALGORITHM:**

STEP 1: Import NumPy using import numpy as np.
STEP 2: Create arrays using np.array() and np.arange().
STEP 3: Perform array operations:

- Reshape arrays
- Slice and index arrays
- Apply mathematical functions (sum, mean, max, min)
- Perform element-wise operations

STEP 4: Display the results of each operation.

**PROGRAM:**

```
# Import NumPy
import numpy as np

# Create a 1D array
arr1 = np.array([10, 20, 30, 40, 50])
print("1D Array:", arr1)

# Create a 2D array using arange and reshape
arr2 = np.arange(1, 10).reshape(3, 3)
print("\n2D Array:\n", arr2)

# Accessing elements
print("\nElement at row 1, column 2:", arr2[0, 1])

# Slicing
print("\nFirst two rows:\n", arr2[:2])

# Mathematical operations
print("\nSum of arr1:", np.sum(arr1))
print("Mean of arr2:", np.mean(arr2))
print("Maximum value in arr2:", np.max(arr2))
print("Minimum value in arr2:", np.min(arr2))

# Element-wise operations
print("\narr1 multiplied by 2:", arr1 * 2)
print("Square of arr2:\n", np.square(arr2))
```

**OUTPUT:**

```
1D Array: [10 20 30 40 50]

2D Array:
 [[1 2 3]
 [4 5 6]
 [7 8 9]]

Element at row 1, column 2: 2

First two rows:
 [[1 2 3]
 [4 5 6]]

Sum of arr1: 150
Mean of arr2: 5.0
Maximum value in arr2: 9
Minimum value in arr2: 1

arr1 multiplied by 2: [ 20  40  60  80 100]
Square of arr2:
 [[ 1  4  9]
 [16 25 36]
 [49 64 81]]
```

**RESULT:**

Thus, the program successfully demonstrates array creation, reshaping, slicing, mathematical computations, and element-wise operations using NumPy.

| EX NO:4 | PROGRAMS TO SOLVE LINEAR ALGEBRA PROBLEMS WITH NUMPY FUNCTIONS |
|---|---|
| DATE: | |

**AIM:**

To solve basic linear algebra problems using NumPy functions in Python, including matrix operations, solving linear equations, and computing transpose, determinant, and inverse of a matrix.

**ALGORITHM:**

STEP 1: Import NumPy using import numpy as np.
STEP 2: Define matrices using np.array().
STEP 3: Perform the following operations:

- Matrix addition
- Matrix multiplication
- Transpose of a matrix
- Determinant of a matrix
- Inverse of a matrix
- Solve a system of linear equations using np.linalg.solve()

STEP 4: Display the results of each operation.

**PROGRAM**

```
# Import NumPy
import numpy as np

# Define matrices
A = np.array([[2, 3],
              [1, 2]])

B = np.array([[1, 0],
              [4, 5]])

# Matrix addition
add_result = A + B
print("Matrix Addition:\n", add_result)

# Matrix multiplication
mul_result = np.dot(A, B)
print("\nMatrix Multiplication:\n", mul_result)

# Transpose of matrix A
transpose_A = A.T
print("\nTranspose of Matrix A:\n", transpose_A)
```

```
# Determinant of matrix A
det_A = np.linalg.det(A)
print("\nDeterminant of Matrix A:", det_A)

# Inverse of matrix A
inverse_A = np.linalg.inv(A)
print("\nInverse of Matrix A:\n", inverse_A)

# Solving linear equations Ax = b
b = np.array([8, 5])
x = np.linalg.solve(A, b)
print("\nSolution of Ax = b:\n", x)
```

**OUTPUT**

```
Matrix Addition:
 [[3 3]
 [5 7]]

Matrix Multiplication:
 [[14 15]
 [ 9 10]]

Transpose of Matrix A:
 [[2 1]
 [3 2]]

Determinant of Matrix A: 1.0

Inverse of Matrix A:
 [[ 2. -3.]
 [-1.  2.]]

Solution of Ax = b:
 [1. 2.]
```

**RESULT:**

Thus, the program successfully demonstrates linear algebra operations using NumPy, including matrix addition, multiplication, transpose, determinant, inverse, and solving a system of linear equations.

| EX NO:5 | PROGRAMS USING PANDAS |
|---|---|
| DATE: | |

**AIM:**

To implement basic data manipulation and analysis using the Pandas library in Python, including DataFrame creation, indexing, filtering, and statistical operations.

**ALGORITHM:**

STEP 1: Import the Pandas library.
STEP 2: Create a DataFrame using a dictionary or a list of lists.
STEP 3: Display the DataFrame.
STEP 4: Perform basic operations:

- Access specific rows and columns.
- Add and delete columns.
- Filter rows based on conditions.
- Perform statistical operations (mean, maximum, minimum).

STEP 5: Print the results of each operation.

**PROGRAM:**

```
# Import pandas

import pandas as pd

# Create a DataFrame

data = {

    'Student': ['Varshini', 'Arun', 'Priya', 'Karthik'],

    'Marks': [85, 92, 78, 88],

    'Department': ['CSE', 'ECE', 'IT', 'CSE']

}

df = pd.DataFrame(data)

# Display the DataFrame

print("Original DataFrame:")

print(df)


# Access specific column

print("\nMarks of all students:")

print(df['Marks'])
```

```
# Add a new column
df['Grade'] = ['B', 'A', 'C', 'B']
print("\nDataFrame after adding 'Grade' column:")
print(df)

# Delete a column
df.drop('Department', axis=1, inplace=True)
print("\nDataFrame after deleting 'Department' column:")
print(df)

# Filter data
print("\nStudents with marks above 80:")
print(df[df['Marks'] > 80])

# Statistical operations
print("\nAverage Marks:", df['Marks'].mean())
print("Maximum Marks:", df['Marks'].max())
print("Minimum Marks:", df['Marks'].min())
```

**OUTPUT:**

```
Original DataFrame:
    Student  Marks Department
0  Varshini     85        CSE
1      Arun     92        ECE
2     Priya     78         IT
3   Karthik     88        CSE

Marks of all students:
0    85
1    92
2    78
3    88
Name: Marks, dtype: int64

DataFrame after adding 'Grade' column:
    Student  Marks Department Grade
0  Varshini     85        CSE     B
1      Arun     92        ECE     A
2     Priya     78         IT     C
3   Karthik     88        CSE     B

DataFrame after deleting 'Department' column:
    Student  Marks Grade
0  Varshini     85     B
1      Arun     92     A
2     Priya     78     C
3   Karthik     88     B

Students with marks above 80:
    Student  Marks Grade
0  Varshini     85     B
1      Arun     92     A
3   Karthik     88     B

Average Marks: 85.75
Maximum Marks: 92
Minimum Marks: 78
```

**RESULT:**

The program successfully demonstrates the use of pandas for data manipulation and analysis.operations such as column access,addition,deletion,filtering and statistical computations were performed and verified.

| EX NO:6 | SOLVING LINEAR ALGEBRA PROBLEMS WITH PANDAS FUNCTIONS |
|---|---|
| DATE: | |

**AIM:**

To solve linear algebra problems using Pandas for structured data representation and NumPy for matrix computations such as addition, multiplication, transpose, determinant, inverse, and solving equations.

**ALGORITHM:**

STEP 1: Import pandas and numpy libraries.
STEP 2: Create matrices using Pandas DataFrames.
STEP 3: Convert DataFrames to NumPy arrays for computation.
STEP 4: Perform linear algebra operations:

- Matrix addition and multiplication
- Transpose, determinant, and inverse.
- Solve a system of linear equations.

STEP 5: Display the results.

**PROGRAM:**

```
# Import libraries
import pandas as pd
import numpy as np

# Create matrices using Pandas DataFrames
df_A = pd.DataFrame([[2, 3], [1, 2]], columns=['A1', 'A2'])
df_B = pd.DataFrame([[1, 0], [4, 5]], columns=['B1', 'B2'])

print("Matrix A:\n", df_A)
print("\nMatrix B:\n", df_B)

# Convert DataFrames to NumPy arrays
A = df_A.to_numpy()
B = df_B.to_numpy()

# Matrix addition
add_result = A + B
print("\nMatrix Addition:\n", add_result)

# Matrix multiplication
mul_result = np.dot(A, B)
print("\nMatrix Multiplication:\n", mul_result)

# Transpose of A
transpose_A = df_A.T
print("\nTranspose of Matrix A:\n", transpose_A)
```

```
# Determinant and inverse of A
det_A = np.linalg.det(A)
inverse_A = np.linalg.inv(A)
print("\nDeterminant of Matrix A:", det_A)
print("\nInverse of Matrix A:\n", pd.DataFrame(inverse_A))

# Solving Ax = b
b = np.array([8, 5])
x = np.linalg.solve(A, b)
print("\nSolution of Ax = b:\n", pd.Series(x, name="x"))
```

**OUTPUT:**

```
Matrix A:
    A1  A2
0   2   3
1   1   2

Matrix B:
    B1  B2
0   1   0
1   4   5

Matrix Addition:
 [[3 3]
 [5 7]]

Matrix Multiplication:
 [[14 15]
 [ 9 10]]

Transpose of Matrix A:
     0  1
A1  2  1
A2  3  2

Determinant of Matrix A: 1.0

Inverse of Matrix A:
      0    1
0  2.0 -3.0
1 -1.0  2.0

Solution of Ax = b:
 0    1.0
1    2.0
Name: x, dtype: float64
```

**RESULT:**

The program successfully demonstrates how Pandas can be used to structure matrix data and how NumPy can perform linear algebra operations. Matrix addition, multiplication, transpose, determinant, inverse, and equation solving were executed and verified.

| EX NO:7 | PROGRAMS FOR READING AND WRITING DATA IN TEXT AND BINARY FORMATS |
|---|---|
| DATE: | |

**AIM:**

To write Python programs that demonstrate reading from and writing to files in both text and binary formats.

**ALGORITHM:**

A. Text File Format

STEP 1: Open a file in write mode using open(filename, 'w').
STEP 2: Write data to the file using write() or writelines().
STEP 3: Close the file.
STEP 4: Open the file in read mode using open(filename, 'r').
STEP 5: Read the contents using read() or readlines() and display the output.

B. Binary File Format

STEP 1: Open a file in binary write mode using open(filename, 'wb').
STEP 2: Convert the data into bytes using encode().
STEP 3: Write the byte data to the file.
STEP 4: Close the file.
STEP 5: Open the file in binary read mode using open(filename, 'rb').
STEP 6: Read the bytes and decode them using decode() to display the content.

**PROGRAM:**

```
# TEXT FORMAT

# Writing to a text file
text_data = "Hello David!\nWelcome to file handling in Python."
with open("sample_text.txt", "w") as text_file:
    text_file.write(text_data)

# Reading from a text file
with open("sample_text.txt", "r") as text_file:
    content = text_file.read()
print("Text File Content:\n", content)

# BINARY FORMAT

# Writing to a binary file
binary_data = text_data.encode('utf-8')  # Convert string to bytes
with open("sample_binary.bin", "wb") as binary_file:
    binary_file.write(binary_data)
```

```
# Reading from a binary file
with open("sample_binary.bin", "rb") as binary_file:
    binary_content = binary_file.read()
print("\nBinary File Content:\n", binary_content.decode('utf-8'))
```

Output

```
Text File Content:
 Hello David!
Welcome to file handling in Python.

Binary File Content:
 Hello David!
Welcome to file handling in Python.
```

**RESULT:**

Thus, the Python program to read from and write data into text and binary files was successfully executed and verified.

| EX NO:8 | PROGRAMS FOR INTERACTING WITH DATABASES |
|---|---|
| DATE: | |

**AIM:**

To write a Python program that connects to a MySQL database, creates a table, inserts records, retrieves data, and displays the results using the MySQL Connector module.

**ALGORITHM:**

STEP 1: Install and import the MySQL. Connector module.
STEP 2: Establish a connection to the MySQL server using host, user, password, and database name.
STEP 3: Create a cursor object to execute SQL queries.
STEP 4: Create a table using the CREATE TABLE statement.
STEP 5: Insert records into the table using INSERT INTO.
STEP 6: Retrieve records using the SELECT statement.
STEP 7: Display the retrieved records.
STEP 8: Close the database connection.

**PROGRAM:**

```
import sqlite3

import pandas as pd

conn=sqlite3.connect("college.db")
cursor = conn.cursor()

cursor.execute("""
CREATE TABLE IF NOT EXISTS students (
   roll INTEGER PRIMARY KEY,
   name TEXT,
   marks INTEGER
)
""")

cursor.execute("INSERT INTO students VALUES (1, 'Varshini', 85)")
cursor.execute("INSERT INTO students VALUES (2, 'Arun', 92)")
cursor.execute("INSERT INTO students VALUES (3, 'Priya', 78)")
conn.commit()

df = pd.read_sql("SELECT * FROM students", conn)
df
conn.close()
```

**OUTPUT:**

| | roll | name | marks |
|---|---|---|---|
| 0 | 1 | Varshini | 85 |
| 1 | 2 | Arun | 92 |
| 2 | 3 | Priya | 78 |

**RESULT:**

Thus, the Python program to interact with a MySQL database by creating a table, inserting records, and retrieving data was successfully executed and verified.

| EX NO:9 | PROGRAMS FOR INTERACTING WITH WEB APIS |
|---|---|
| DATE: | |

**AIM:**

To write a Python program that interacts with a public Web API using the requests module to fetch and display data.

**ALGORITHM**

STEP 1. Import the requests module.
STEP 2. Choose a public API endpoint (for example, JSONPlaceholder).
STEP 3. Send a GET request to the API using requests.get().
STEP 4. Check the response status code.
STEP 5. Parse the JSON response using the json() method.
STEP 6. Extract and display relevant user data.

**PROGRAM:**

```
# Import requests module
import requests

# Define the API endpoint
url = "https://jsonplaceholder.typicode.com/users"

# Send GET request
response = requests.get(url)

# Check status code
if response.status_code == 200:
    # Parse JSON response
    users = response.json()

    # Display user data
    print("User Details from API:\n")
    for user in users:
        print(f"Name: {user['name']}, Email: {user['email']}, City: {user['address']['city']}")
else:
    print("Failed to retrieve data. Status code:", response.status_code)
```

**OUTPUT:**

```
User Details from API:

Name: Leanne Graham, Email: Sincere@april.biz, City: Gwenborough
Name: Ervin Howell, Email: Shanna@melissa.tv, City: Wisokyburgh
Name: Clementine Bauch, Email: Nathan@yesenia.net, City: McKenziehaven
Name: Patricia Lebsack, Email: Julianne.OConner@kory.org, City: South Elvis
Name: Chelsey Dietrich, Email: Lucio_Hettinger@annie.ca, City: Roscoeview
Name: Mrs. Dennis Schulist, Email: Karley_Dach@jasper.info, City: South Christy
Name: Kurtis Weissnat, Email: Telly.Hoeger@billy.biz, City: Howemouth
Name: Nicholas Runolfsdottir V, Email: Sherwood@rosamond.me, City: Aliyaview
Name: Glenna Reichert, Email: Chaim_McDermott@dana.io, City: Bartholomebury
Name: Clementina DuBuque, Email: Rey.Padberg@karina.biz, City: Lebsackbury
```

**RESULT:**

Thus, the Python program to interact with a public Web API, retrieve JSON data, and display user details was successfully executed and verified.

| EX NO:10 | PROGRAMS FOR STRING MANIPULATIONS |
|---|---|
| DATE: | |

**AIM:**

To write Python programs that demonstrate various string manipulation techniques such as slicing, concatenation, case conversion, searching, and replacing.

**ALGORITHM:**

STEP 1: Accept a string input from the user.
STEP 2: Perform the following operations:

- Length of the string using len().
- Convert to uppercase and lowercase using .upper() and .lower().
- Reverse the string using slicing.
- Concatenate with another string.
- Search for a substring using .find() or in.
- Replace a word using .replace().
- Split the string using .split().

STEP 3: Display the results of each operation.

**PROGRAM:**

```
# Accept input
text = input("Enter a string: ")

# Length of the string
print("\nLength of the string:", len(text))

# Convert to uppercase and lowercase
print("Uppercase:", text.upper())
print("Lowercase:", text.lower())

# Reverse the string
print("Reversed string:", text[::-1])

# Concatenate with another string
extra = " - Python Lab"
print("Concatenated string:", text + extra)

# Search for a substring
search_term = "lab"
if search_term in text.lower():
    print(f"'{search_term}' found in the string.")
else:
```

```
    print(f"'{search_term}' not found in the string.")

# Replace a word
replaced_text = text.replace("Python", "Programming")
print("Replaced string:", replaced_text)

# Split the string
print("Split string:", text.split())
```

**OUTPUT:**

```
Enter a string:  WELCOME TO PYTHON LAB

Length of the string: 21
Uppercase: WELCOME TO PYTHON LAB
Lowercase: welcome to python lab
Reversed string: BAL NOHTYP OT EMOCLEW
Concatenated string: WELCOME TO PYTHON LAB - Python Lab
'lab' found in the string.
Replaced string: WELCOME TO PYTHON LAB
Split string: ['WELCOME', 'TO', 'PYTHON', 'LAB']
```

**RESULT:**

The program successfully demonstrates various string manipulation techniques in python,including slicing,case conversion,searching,replacing and formatting.

| EX NO:11 | DATA WRANGLING |
|---|---|
| DATE: | |

**AIM:**

To perform data wrangling operations using Python and pandas, including loading, cleaning, transforming, and filtering data for analysis.

**ALGORITHM:**

STEP 1: Import pandas using import pandas as pd.
STEP 2: Load data from a CSV file or dictionary into a DataFrame.
STEP 3: Perform wrangling operations:

- Handle missing values using fillna() or dropna().
- Rename columns using rename().
- Filter rows based on conditions.
- Convert data types using astype().
- Create new columns using expressions.

STEP 4: Display the cleaned and transformed data.

**PROGRAM:**

```
# Import pandas
import pandas as pd

# Create sample data with missing values
data = {
    'Name': ['Varshini', 'Arun', 'Priya', None],
    'Age': [21, 22, None, 20],
    'Marks': [85, 92, 78, None]
}

# Create DataFrame
df = pd.DataFrame(data)
print("Original Data:\n", df)

# Handle missing values
df['Name'] = df['Name'].fillna('Unknown')
df['Age'] = df['Age'].fillna(df['Age'].mean())
df['Marks'] = df['Marks'].fillna(df['Marks'].mean())

# Rename column
df = df.rename(columns={'Marks': 'Score'})

# Convert Age to integer
df['Age'] = df['Age'].astype(int)
```

```
# Create new column 'Grade'
df['Grade'] = ['B', 'A', 'C', 'B']

# Filter rows where Score > 80
filtered_df = df[df['Score'] > 80]

print("\nCleaned and Transformed Data:\n", df)
print("\nFiltered Data (Score > 80):\n", filtered_df)
```

**OUTPUT:**

```
Original Data:
        Name   Age  Marks
0  Varshini  21.0   85.0
1      Arun  22.0   92.0
2     Priya   NaN   78.0
3      None  20.0    NaN

Cleaned and Transformed Data:
        Name  Age  Score Grade
0  Varshini   21   85.0     B
1      Arun   22   92.0     A
2     Priya   21   78.0     C
3   Unknown   20   85.0     B

Filtered Data (Score > 80):
        Name  Age  Score Grade
0  Varshini   21   85.0     B
1      Arun   22   92.0     A
3   Unknown   20   85.0     B
```

**RESULT:**

The program successfully demonstrates data wrangling using pandas.it handles missing values, renames columns, filters data, converts types and creates row columns for analysis.

| EX NO:12 | DATA AGGREGATION OPERATIONS |
|---|---|
| DATE: | |

**AIM:**

To perform data aggregation operations using Python and pandas, including grouping, summarizing, and computing aggregate statistics such as sum, mean, count, and max.

**ALGORITHM:**

STEP 1: Import pandas using import pandas as pd.
STEP 2: Create a Data Frame with sample data.
STEP 3: Use groupby() to group data based on one or more columns.
STEP 4: Apply aggregation functions like sum(), mean(), count(), max(), min() on grouped data.
STEP 5: Display the aggregated results.

**PROGRAM:**

```
# Import pandas
import pandas as pd

# Create sample data
data = {
    'Department': ['CSE', 'ECE', 'CSE', 'IT', 'ECE', 'IT'],
    'Student': ['Varshini', 'Arun', 'Priya', 'Karthik', 'Divya', 'Ravi'],
    'Marks': [85, 92, 78, 88, 75, 80]
}

df = pd.DataFrame(data)
print("Original Data:\n", df)

# Group by Department and aggregate
grouped = df.groupby('Department')

# Apply aggregation functions
print("\nTotal Marks by Department:\n", grouped['Marks'].sum())
print("\nAverage Marks by Department:\n", grouped['Marks'].mean())
print("\nMaximum Marks by Department:\n", grouped['Marks'].max())
print("\nStudent Count by Department:\n", grouped['Student'].count())
```

**OUTPUT:**

```
Original Data:
   Department   Student  Marks
0         CSE  Varshini     85
1         ECE      Arun     92
2         CSE     Priya     78
3          IT   Karthik     88
4         ECE     Divya     75
5          IT      Ravi     80

Total Marks by Department:
 Department
CSE    163
ECE    167
IT     168
Name: Marks, dtype: int64

Average Marks by Department:
 Department
CSE    81.5
ECE    83.5
IT     84.0
Name: Marks, dtype: float64

Maximum Marks by Department:
 Department
CSE    85
ECE    92
IT     88
Name: Marks, dtype: int64

Student Count by Department:
 Department
CSE    2
ECE    2
IT     2
Name: Student, dtype: int64
```

**RESULT:**

The program successfully demonstrates data aggregation operations using pandas,grouping and summarizing techniques such as sum,mean,max and count are applied and verified.

| EX NO:13 | HANDLE TIME SERIES DATA |
|---|---|
| DATE: | |

**AIM:**

To demonstrate how to handle time series data using Python and pandas by creating, indexing, resampling, and performing basic analysis on datetime-indexed data.

**ALGORITHM:**

STEP 1: Import pandas and create a time series using date_range() and Series().
STEP 2: Set the datetime index.
STEP 3: Filter data by date range.
STEP 4: Resample data to a new frequency and compute mean.
STEP 5: Apply rolling window operations.
STEP 6: Display results using DataFrames.

**PROGRAM:**

```
import pandas as pd

# Create time series data
dates = pd.date_range(start='2023-01-01', periods=10, freq='D')
values = pd.Series([10, 12, 15, 14, 13, 16, 18, 17, 19, 20], index=dates)

# Display original time series as table
df_original = pd.DataFrame({'Date': dates, 'Value': values.values})
print("\nOriginal Time Series Data:")
print(df_original)

# Filter data by date range
filtered = values['2023-01-03':'2023-01-07']
df_filtered = pd.DataFrame({'Date': filtered.index, 'Filtered Value': filtered.values})
print("\nFiltered Data (2023-01-03 to 2023-01-07):")
print(df_filtered)

# Resample to 5-day frequency and compute mean
resampled = values.resample('5D').mean()
df_resampled = pd.DataFrame({
    'Period Start': resampled.index,
    '5-Day Mean': resampled.values
})
print("\nResampled Data (5-Day Mean):")
print(df_resampled)

# Rolling mean with window=3
rolling_mean = values.rolling(window=3).mean()
df_rolling = pd.DataFrame({
```

```
    'Date': rolling_mean.index,
    'Rolling Mean (3-day)': rolling_mean.values
})
print("\nRolling Mean (Window=3):")
print(df_rolling)
```

**OUTPUT:**

```
Original Time Series Data:
        Date  Value
0 2023-01-01     10
1 2023-01-02     12
2 2023-01-03     15
3 2023-01-04     14
4 2023-01-05     13
5 2023-01-06     16
6 2023-01-07     18
7 2023-01-08     17
8 2023-01-09     19
9 2023-01-10     20

Filtered Data (2023-01-03 to 2023-01-07):
        Date  Filtered Value
0 2023-01-03              15
1 2023-01-04              14
2 2023-01-05              13
3 2023-01-06              16
4 2023-01-07              18

Resampled Data (5-Day Mean):
  Period Start  5-Day Mean
0   2023-01-01        12.8
1   2023-01-06        18.0

Rolling Mean (Window=3):
        Date  Rolling Mean (3-day)
0 2023-01-01                   NaN
1 2023-01-02                   NaN
2 2023-01-03             12.333333
3 2023-01-04             13.666667
4 2023-01-05             14.000000
5 2023-01-06             14.333333
6 2023-01-07             15.666667
7 2023-01-08             17.000000
8 2023-01-09             18.000000
9 2023-01-10             18.666667
```

**RESULT:**

The program successfully demonstrates time series data using Python and pandas by creating, indexing, resampling, and performing basic analysis on date time-indexed data.

| EX NO:14 | VISUALIZATION OF DIFFERENT KINDS OF DATA |
|---|---|
| DATE: | |

**AIM:**

To visualize different types of data using Python libraries such as Matplotlib and Pandas, including bar charts, line charts, and pie charts for better data interpretation.

**ALGORITHM:**

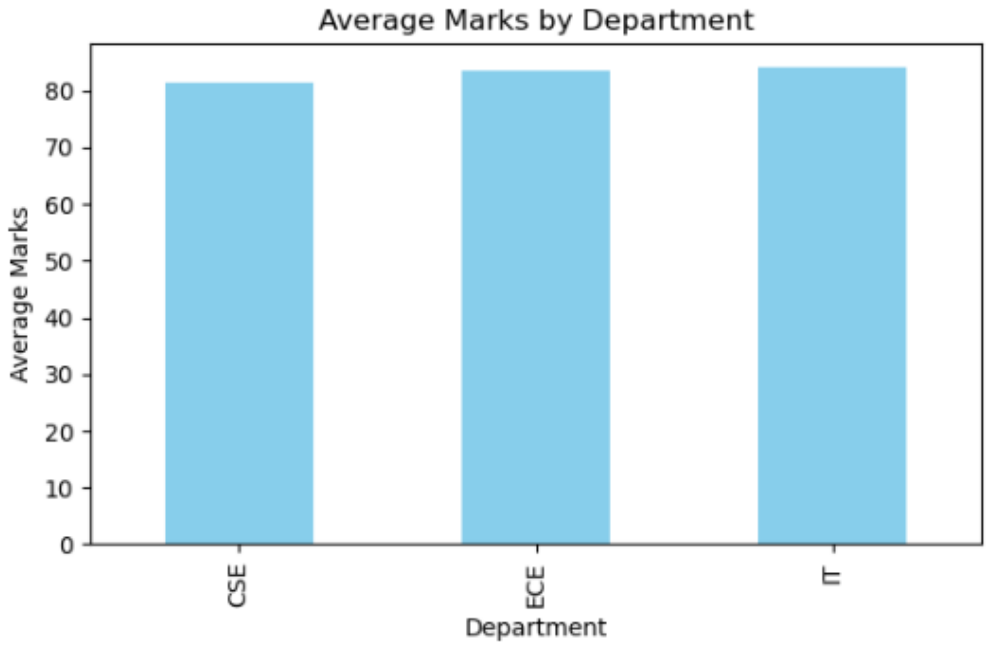
STEP 1: Import necessary libraries: pandas and matplotlib.pyplot.
STEP 2: Create a dataset using a dictionary and convert it into a DataFrame.
STEP 3: Generate visualizations:
STEP 4: Bar chart for department-wise average marks.
STEP 5: Line chart for student scores over time.
STEP 6: Pie chart for department-wise student distribution.
STEP 7: Customize plots with titles, labels, and legends.
STEP 8: Display or save the plots.

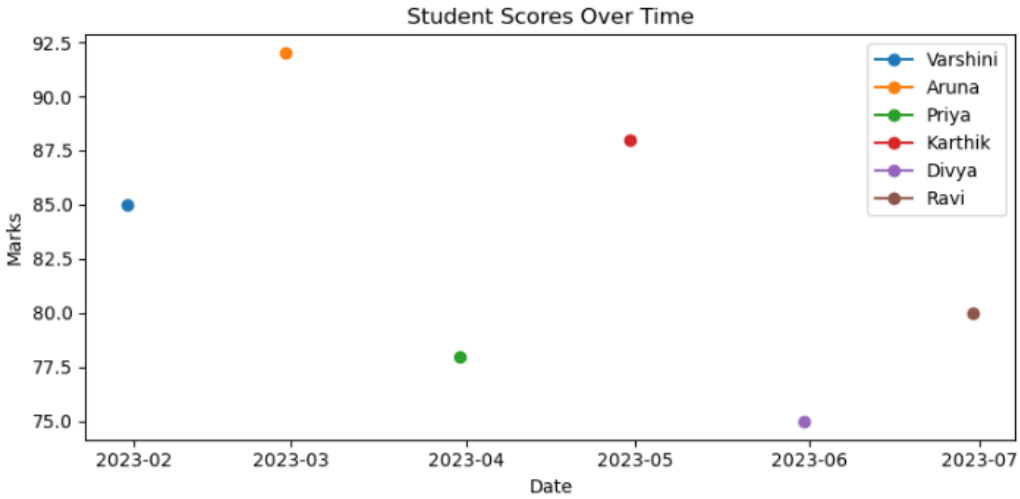
**PROGRAM:**

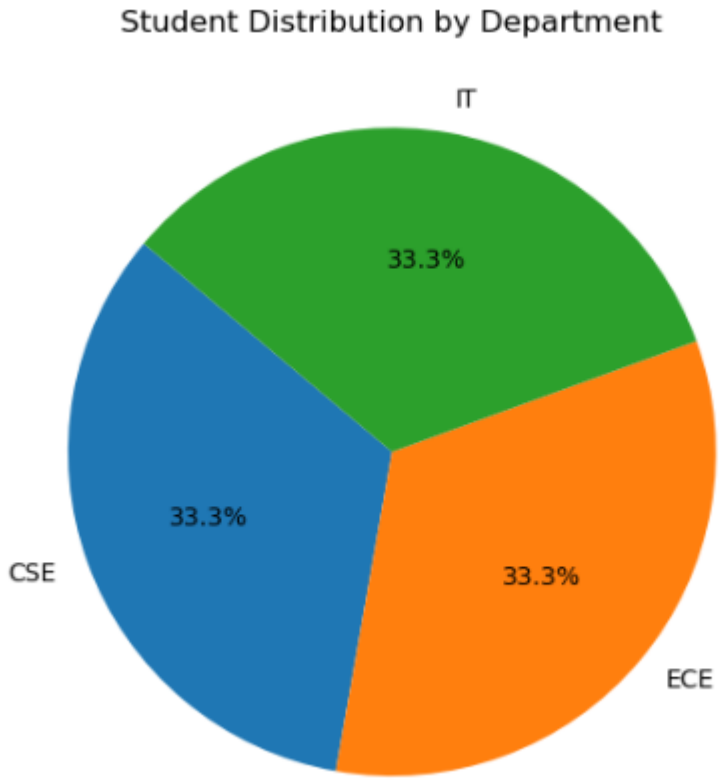
```
import pandas as pd
import matplotlib.pyplot as plt

# Sample data
data = {
    'Student': ['Varshini', 'Aruna', 'Priya', 'Karthik', 'Divya', 'Ravi'],
    'Department': ['CSE', 'ECE', 'CSE', 'IT', 'ECE', 'IT'],
    'Marks': [85, 92, 78, 88, 75, 80],
    'Date': pd.date_range(start='2023-01-01', periods=6, freq='ME')
}

# Create DataFrame
df = pd.DataFrame(data)

# -------------------------------
# Bar Chart: Average Marks by Department
# -------------------------------
avg_marks = df.groupby('Department')['Marks'].mean()

plt.figure(figsize=(6, 4))
avg_marks.plot(kind='bar')
plt.title('Average Marks by Department')
plt.xlabel('Department')
plt.ylabel('Average Marks')
plt.tight_layout()
plt.show()

# -------------------------------
```

```
# Line Chart: Student Scores Over Time
# --------------------------------
plt.figure(figsize=(8, 4))
plt.plot(df['Date'], df['Marks'], marker='o')

plt.title('Student Scores Over Time')
plt.xlabel('Date')
plt.ylabel('Marks')
plt.tight_layout()
plt.show()

# --------------------------------
# Pie Chart: Student Distribution by Department
# --------------------------------
dept_counts = df['Department'].value_counts()

plt.figure(figsize=(5, 5))
plt.pie(dept_counts, labels=dept_counts.index, autopct='%1.1f%%', startangle=140)
plt.title('Student Distribution by Department')
plt.tight_layout()
plt.show()
```

**OUTPUT:**






**RESULT:**

Thus, different kinds of data were successfully visualized using Python libraries such as Pandas and Matplotlib by generating bar charts, line charts, and pie charts for effective data interpretation.

| EX NO:15 | DISTRIBUTION ANALYSIS OF NUMERICAL DATA |
|---|---|
| DATE: | |

**AIM:**

To analyze the distribution of numerical data using Python and visualize it with histograms, box plots, and density plots using Pandas, Matplotlib, and Seaborn.

**ALGORITHM:**

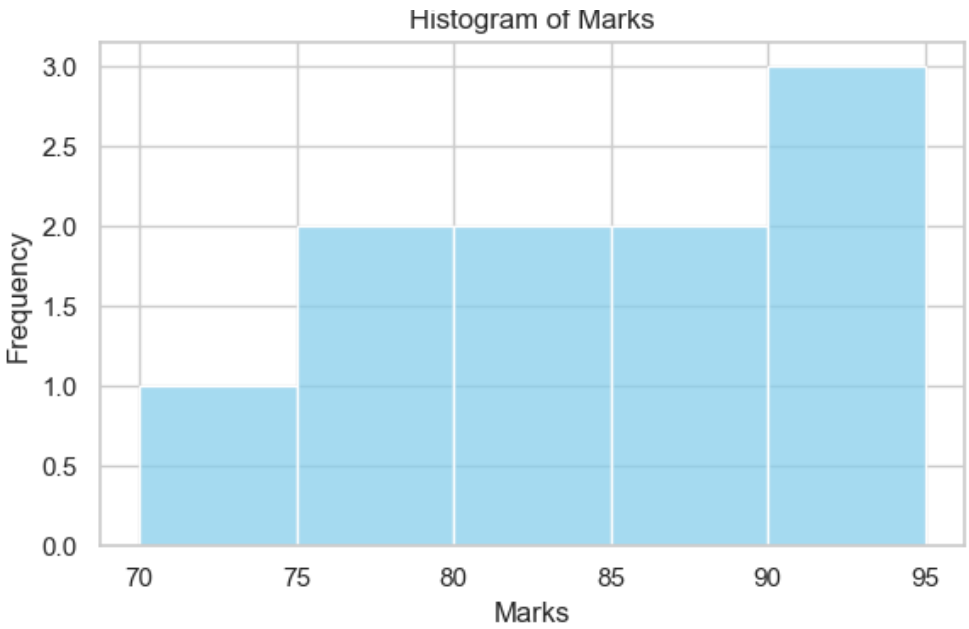
STEP 1. Import necessary libraries: pandas, matplotlib.pyplot, and seaborn.
STEP 2. Create or load a dataset containing numerical values.
STEP 3. Use Pandas and Seaborn to perform distribution analysis:
STEP 4. Generate a histogram to show frequency distribution.
STEP 5. Generate a box plot to show data spread and outliers.
STEP 6. Generate a density plot to show probability distribution.
STEP 7. Customize plots with appropriate titles and axis labels.
STEP 8. Display the plots.

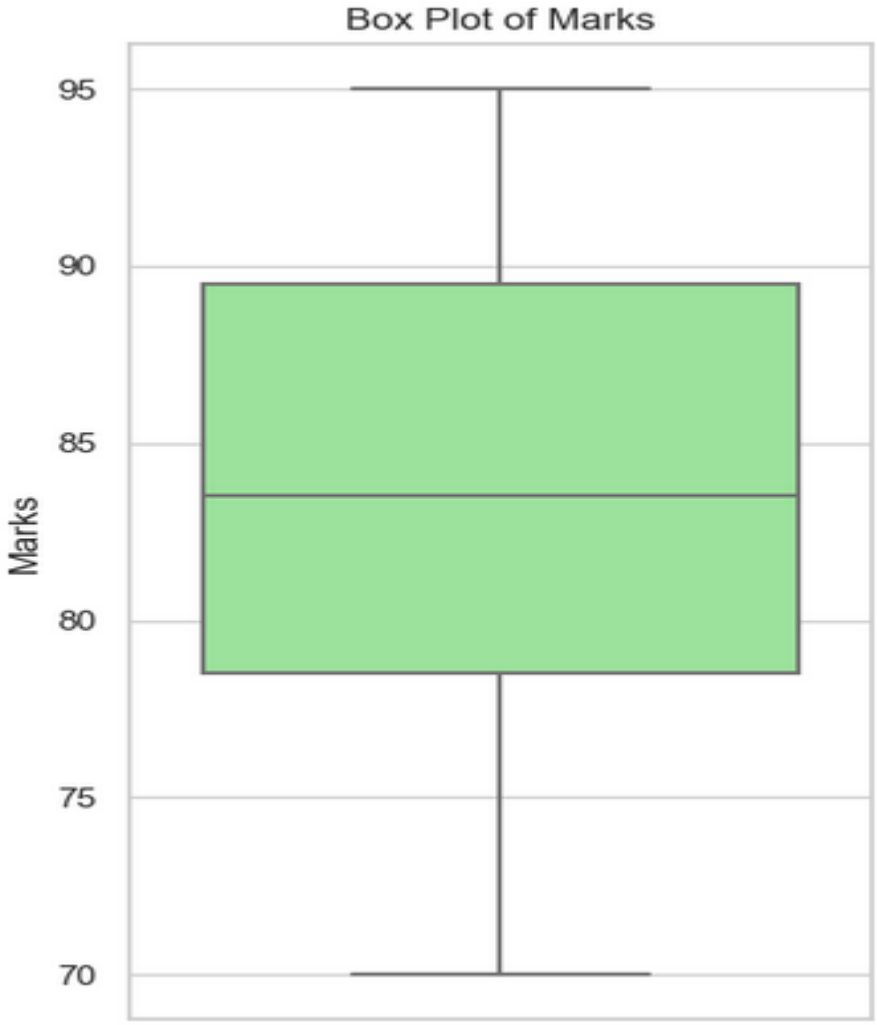
**PROGRAM:**

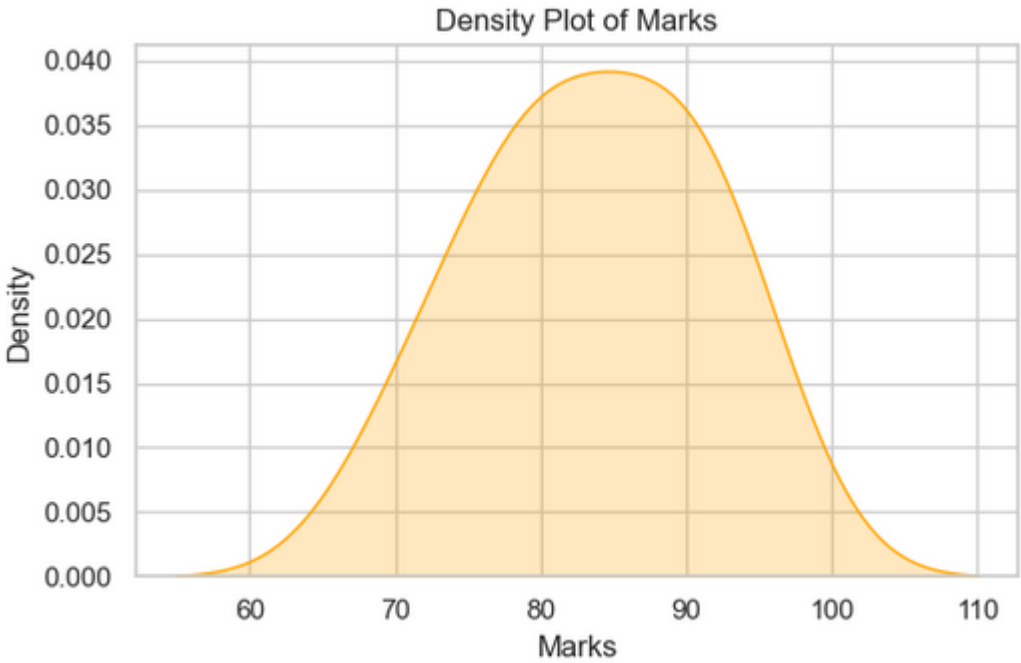
```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Set seaborn style
sns.set(style="whitegrid")

# Create sample data
df = pd.DataFrame({
    "Marks": [85, 92, 78, 88, 75, 80, 95, 70, 82, 90]
})

# Histogram
plt.figure(figsize=(6, 4))
sns.histplot(df["Marks"], bins=5, color="skyblue")
plt.title("Histogram of Marks")
plt.xlabel("Marks")
plt.ylabel("Frequency")
plt.tight_layout()
plt.show()

# Box Plot
plt.figure(figsize=(4, 6))
sns.boxplot(y=df["Marks"], color="lightgreen")
plt.title("Box Plot of Marks")
plt.ylabel("Marks")
plt.tight_layout()
plt.show()
```

```
# Density Plot
plt.figure(figsize=(6, 4))
sns.kdeplot(df["Marks"], fill=True, color="orange")
plt.title("Density Plot of Marks")
plt.xlabel("Marks")
plt.tight_layout()
plt.show()
```

**OUTPUT:**




Box Plot of Marks
Marks
95
90
85
80
75
70




**RESULT:**

Thus, the distribution of numerical data was successfully analysed using Python. Histograms, box plots, and density plots were generated using Pandas, Matplotlib, and Seaborn to visualize frequency distribution, data spread, and probability distribution effectively.